Lecture # 7

By: Muhammad Zain Uddin

email: zuddin@iba.edu.pk

Agenda

OpenMP

Parallel Directive

Work-Sharing Directives

Attribution

TODAY'S LECTURE IS BASED ON LAWRENCE LIVERMORE NATIONAL LABORATORY'S HPC TUTORIALS

What is OpenMP

An Application Program Interface (API) that may be used to explicitly direct ***multi-threaded, shared memory parallelism***

- Comprised of three primary API components:
  - Compiler Directives
  - Runtime Library Routines
  - Environment Variables

- An abbreviation for:
  - Short version: **Open Multi-Processing**
  - Long version: **Open** specifications for **Multi-Processing** via collaborative work between interested parties from the hardware and software industry, government and academia.

OpenMP is NOT

- Necessarily implemented identically by all vendors
- Guaranteed to make the most efficient use of shared memory
- Required to check for data dependencies, data conflicts, race conditions, or deadlocks
- Required to check for code sequences that cause a program to be classified as non-conforming
- Designed to guarantee that input or output to the same file is synchronous when executed in parallel. The programmer is responsible for synchronizing input and output.

Goals of OpenMP

- **Standardization:**
  - Provide a standard among a variety of shared memory architectures/platforms
  - Jointly defined and endorsed by a group of major computer hardware and software vendors
- **Lean and Mean:**
  - Establish a simple and limited set of directives for programming shared memory machines.
  - Significant parallelism can be implemented by using just 3 or 4 directives.
  - This goal is becoming less meaningful with each new release, apparently.

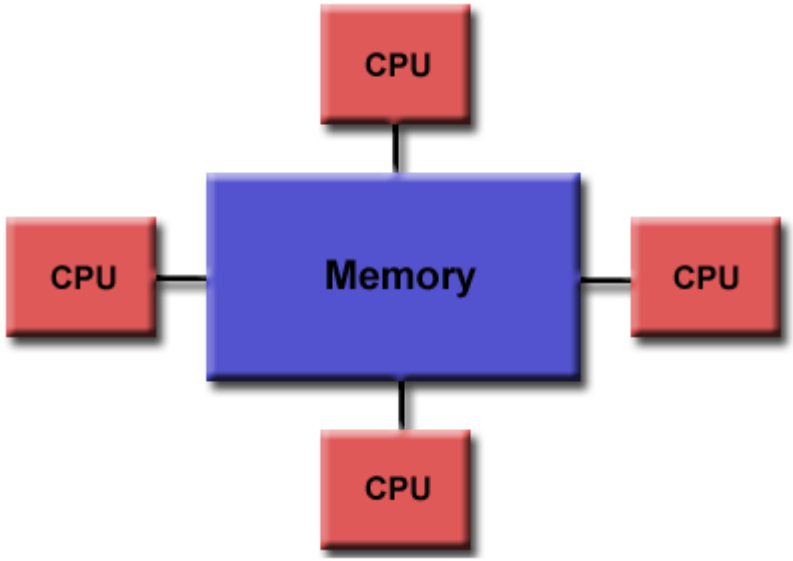
Goals of OpenMP

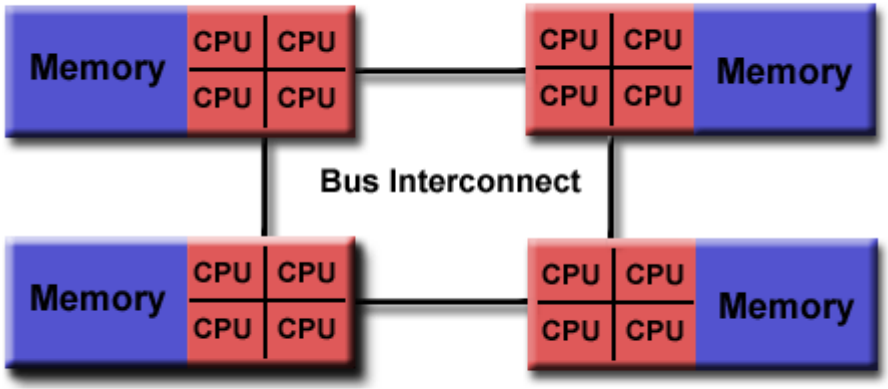
- **Ease of Use:**
  - Provide capability to incrementally parallelize a serial program, unlike message-passing libraries which typically require an all or nothing approach
  - Provide the capability to implement both coarse-grain and fine-grain parallelism
- **Portability:**
  - The API is specified for C/C++ and Fortran
  - Public forum for API and membership
  - Most major platforms have been implemented including Unix/Linux platforms and Windows

OpenMP Programming Model

OpenMP is designed for multi-processor/core, shared memory machines.

The underlying architecture can be shared memory UMA or NUMA.





OpenMP Programming Model

OpenMP programs accomplish parallelism exclusively through the use of **threads**.

- A thread of execution is the smallest unit of processing that can be scheduled by an operating system.
- Threads exist within the resources of a single process. Without the process, they cease to exist.

Typically, the number of threads match the number of machine processors/cores. However, the actual use of threads is up to the application.

Example

```c
#include<stdio.h>
#include<omp.h>

void main()
{
    #pragma omp parallel
    {
        printf("Hello world!\n");
    }
    return;
}
```

```
Hello world!
Hello world!
```

Explicit Parallelism

OpenMP is an **explicit** (not automatic) programming model, offering the programmer full control over parallelization.

Parallelization can be as simple as taking a serial program and inserting compiler directives

Or

As complex as inserting subroutines to set multiple levels of parallelism, locks and even nested locks.

Example 2

```c
#include<stdio.h>
#include<omp.h>

void main()
{
   omp_set_num_threads(4);
   #pragma omp parallel
   {
      printf("Hello world!\n");
   }
   return;
}
```

```c
#include<stdio.h>
#include<omp.h>

void main()
{
   omp_set_num_threads(4);
   int id;
   #pragma omp parallel
   {
      id = omp_get_thread_num();
      printf("Thread %d: Hello world!\n", id);
   }
   return;
}
```

```
Thread 0: Hello world!
Thread 2: Hello world!
Thread 1: Hello world!
Thread 3: Hello world!
```

```
Thread 3: Hello world!
Thread 0: Hello world!
Thread 2: Hello world!
Thread 1: Hello world!
```

Fork-Join Model

OpenMP uses the fork-join model of parallel execution

All OpenMP programs begin as a single process: the **master thread**.

- The master thread executes sequentially until the first **parallel region** construct is encountered.

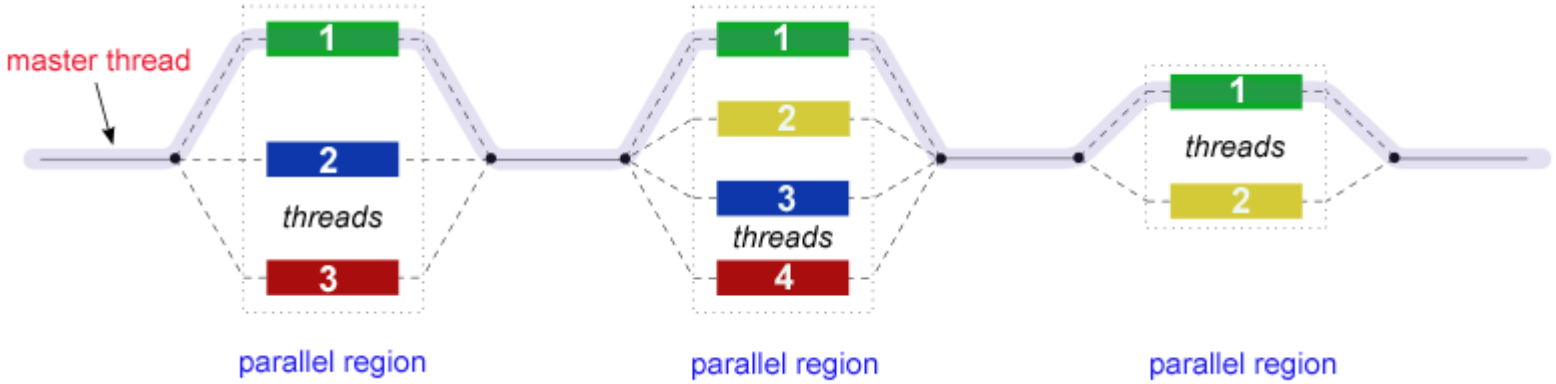
FORK: the master thread then creates a team of parallel *threads*.

- The statements in the program that are enclosed by the parallel region construct are then executed in parallel among the various team threads.

JOIN: When the team threads complete the statements in the parallel region construct, they synchronize and terminate, leaving only the master thread.

- The number of parallel regions and the threads that comprise them are arbitrary.

Fork-Join Model



OpenMP Execution Model

Most OpenMP parallelism is specified through the use of compiler directives which are imbedded in C/C++ or Fortran source code.

The API provides for the placement of parallel regions inside other parallel regions.

- Implementations may or may not support this feature.

The API provides for the runtime environment to dynamically alter the number of threads used to execute parallel regions.

OpenMP specifies nothing about parallel I/O.

- It is entirely up to the programmer to ensure that I/O is conducted correctly within the context of a multi-threaded program.

OpenMP Execution Model

Single-Program-Multiple-Data (SPMD) is underlying programming paradigm

- all threads have potential to execute the same program code,
- However, each thread may access/modify different data and traverse different execution paths.

OpenMP provides a “relaxed-consistency” and “temporary” view of thread memory

- threads have equal access to shared memory where variables can be retrieved/stored.
- Each thread also has its own temporary copies of variables that may be modified independent from variables in memory.

Compiler Directives

Compiler directives appear as comments in your source code and are ignored by compilers

- unless you tell them otherwise - usually by specifying the appropriate compiler flag

OpenMP compiler directives are used for various purposes:

- Spawning a parallel region
- Dividing blocks of code among threads
- Distributing loop iterations between threads
- Serializing sections of code
- Synchronization of work among threads

Runtime Library Routines

The OpenMP API includes an ever-growing number of run-time library routines.

These routines are used for a variety of purposes:

- Setting and querying the number of threads
- Querying a thread's unique identifier (thread ID), a thread's ancestor's identifier, the thread team size
- Setting and querying the dynamic threads feature
- Querying if in a parallel region, and at what level
- Setting and querying nested parallelism
- Setting, initializing and terminating locks and nested locks
- Querying wall clock time and resolution

Environment Variables

OpenMP provides several environment variables for controlling the execution of parallel code at run-time.

These environment variables can be used to control such things as:

- Setting the number of threads
- Specifying how loop interations are divided
- Binding threads to processors
- Enabling/disabling nested parallelism; setting the maximum levels of nested parallelism
- Enabling/disabling dynamic threads
- Setting thread stack size
- Setting thread wait policy

OpenMP Directive Format (C/C++)

Format:

#pragma omp	directive-name	[clause, ...]	newline
Required for all OpenMP C/C++ directives.	A valid OpenMP directive must appear after the pragma and before any clauses.	Optional. Clauses can be in any order, and repeated as necessary unless otherwise restricted.	Required. Precedes the structured block which is enclosed by this directive.

Example:

```
#pragma omp parallel default(shared) private(beta,pi)
```

- Case sensitive.
- Directives follow conventions of the C/C++ standards for compiler directives.
- Only one directive-name may be specified per directive.
- Each directive applies to at most one succeeding statement, which must be a structured block.
- Long directive lines can be “continued” on succeeding lines by escaping the newline character with a backslash (“ ") at the end of a directive line.

Parallel Region Construct

A **parallel** region is a block of code that will be executed by multiple threads.

When a thread reaches a PARALLEL directive, it creates a team of threads and becomes the master of the team.

- The master is a member of that team and has thread number 0 within that team.

Starting from the beginning of this parallel region, the code is duplicated and all threads will execute that code.

There is an implied barrier at the end of a parallel section.

- Only the master thread continues execution past this point.

If any thread terminates within a parallel region, all threads in the team will terminate, and the work done up until that point is undefined.

Sample Code: Parallel Directive

```
#include <omp.h>

main ()  {

int nthreads, tid;

/* Fork a team of threads with each thread having a private tid variable */
#pragma omp parallel private(tid)
  {

  /* Obtain and print thread id */
  tid = omp_get_thread_num();
  printf("Hello World from thread = %d\n", tid);

  /* Only master thread does this */
  if (tid == 0)
    {
    nthreads = omp_get_num_threads();
    printf("Number of threads = %d\n", nthreads);
    }

  }  /* All threads join master thread and terminate */

}
```

How many threads?

The number of threads in a parallel region is determined by the following factors, in order of precedence:

- Evaluation of the IF clause
- Setting of the NUM_THREADS clause
- Use of the omp_set_num_threads() library function
- Setting of the OMP_NUM_THREADS environment variable
- Implementation default - usually the number of CPUs on a node, though it could be dynamic.
- Threads are numbered from 0 (master thread) to N-1.

Restrictions

A parallel region must be a structured block that does not span multiple routines or code files.

It is illegal to branch (goto) into or out of a parallel region.

Only a single IF clause is permitted.

Only a single NUM_THREADS clause is permitted.

A program must not depend upon the ordering of the clauses.

Work-Sharing Constructs

A work-sharing construct divides the execution of the enclosed code region among the members of the team that encounter it.

Work-sharing constructs do not launch new threads

There is no implied barrier upon entry to a work-sharing construct, however there is an implied barrier at the end of a work sharing construct.

Restrictions

A work-sharing construct must be enclosed dynamically within a parallel region in order for the directive to execute in parallel.

Work-sharing constructs must be encountered by all members of a team or none at all.

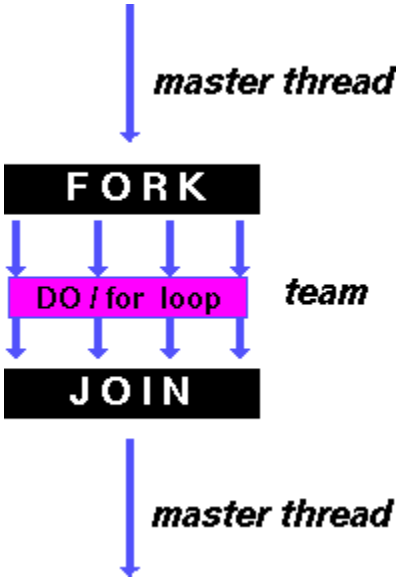
Successive work-sharing constructs must be encountered in the same order by all members of a team.

Types of Work-Sharing Constructs

DO / for

DO / for shares iterations of a loop across the team.

- Represents a type of “data parallelism”.



DO/for Work Sharing Construct

SCHEDULE clause describes how iterations of the loop are divided among the threads in the team.

- Static
- Dynamic
- Guided
- Runtime

NO WAIT clause If specified, threads do not synchronize at the end of the parallel loop.

ORDERED clause specifies that the iterations of the loop must be executed as they would be in a serial program.

DO/for Restrictions

The DO loop can not be a DO WHILE loop, or a loop without loop control.

- Also, the loop iteration variable must be an integer and the loop control parameters must be the same for all threads.

Program correctness must not depend upon which thread executes a particular iteration.

It is illegal to branch (goto) out of a loop associated with a DO/for directive.

The chunk size must be specified as a loop invariant integer expression, as there is no synchronization during its evaluation by different threads.

ORDERED and SCHEDULE clauses may appear once each.

Sample Code

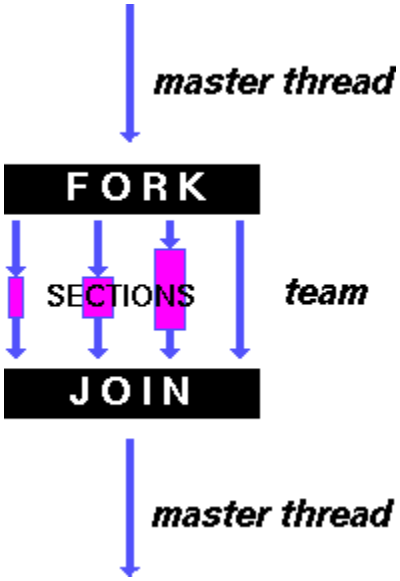
```
#include <omp.h>
#define CHUNKSIZE 100
#define N     1000

main ()
{

int i, chunk;
float a[N], b[N], c[N];

/* Some initializations */
for (i=0; i < N; i++)
  a[i] = b[i] = i * 1.0;
chunk = CHUNKSIZE;

#pragma omp parallel shared(a,b,c,chunk) private(i)
  {

  #pragma omp for schedule(dynamic,chunk) nowait
  for (i=0; i < N; i++)
    c[i] = a[i] + b[i];

  }  /* end of parallel section */

}
```

Types of Work-Sharing Constructs

SECTIONS

Sections breaks work into separate, discrete sections. Each section is executed by a thread.

- Can be used to implement a type of “functional parallelism”.



Section Work-Sharing Construct

The SECTIONS directive is a non-iterative work-sharing construct.

- It specifies that the enclosed section(s) of code are to be divided among the threads in the team.

Independent SECTION directives are nested within a SECTIONS directive. Each SECTION is executed once by a thread in the team.

- Different sections may be executed by different threads.
- It is possible for a thread to execute more than one section if it is quick enough and the implementation permits such.

There is an implied barrier at the end of a SECTIONS directive, unless the NOWAIT clause is used.

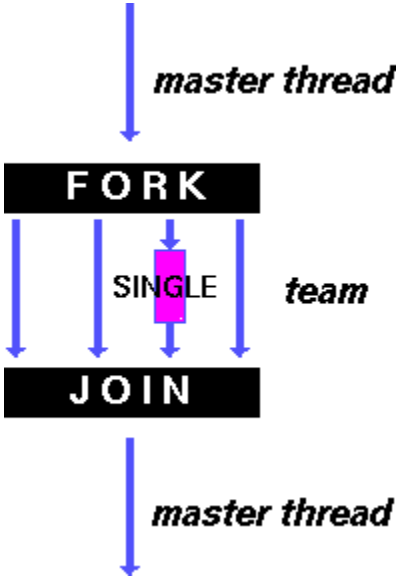
Sample Code

```
#include <omp.h>
#define N     1000

main ()
{
int i;
float a[N], b[N], c[N], d[N];

/* Some initializations */
for (i=0; i < N; i++) {
  a[i] = i * 1.5;
  b[i] = i + 22.35;
  }

#pragma omp parallel shared(a,b,c,d) private(i)
  {
  #pragma omp sections nowait
    {
    #pragma omp section
    for (i=0; i < N; i++)
      c[i] = a[i] + b[i];

    #pragma omp section
    for (i=0; i < N; i++)
      d[i] = a[i] * b[i];

    }  /* end of sections */
  }  /* end of parallel section */
}
```

Types of Work-Sharing Constructs

SINGLE

SINGLE serializes a section of code



Single Work-Sharing Construct

The SINGLE directive specifies that the enclosed code is to be executed by only one thread in the team.

Useful when dealing with sections of code that are not thread safe (such as I/O)

Threads in the team that do not execute the SINGLE directive, wait at the end of the enclosed code block, unless a NOWAIT/nowait clause is specified.

It is illegal to branch into or out of a SINGLE block.

Sample Code

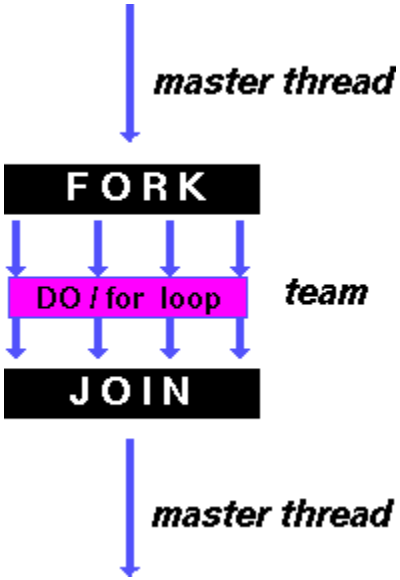
```c
#include <stdio.h>
#include <omp.h>

int main() {
    omp_set_num_threads(4); // Set number of threads

    #pragma omp parallel
    {
        int thread_id = omp_get_thread_num();

        // Each thread prints its ID
        printf("Thread %d: Before SINGLE\n", thread_id);

        // Only one thread executes this block
        #pragma omp single
        {
            printf("Thread %d: Executing SINGLE block\n", thread_id);
        }

        // All threads wait at the implicit barrier (unless nowait is used)
        printf("Thread %d: After SINGLE\n", thread_id);
    }

    return 0;
```

Types of Work-Sharing Constructs

DO / for

DO / for shares iterations of a loop across the team.

- Represents a type of “data parallelism”.



DO/for Work Sharing Construct

SCHEDULE clause describes how iterations of the loop are divided among the threads in the team.

- Static
- Dynamic
- Guided
- Runtime

NO WAIT clause If specified, threads do not synchronize at the end of the parallel loop.

ORDERED clause specifies that the iterations of the loop must be executed as they would be in a serial program.

DO/for Restrictions

The DO loop can not be a DO WHILE loop, or a loop without loop control.

- Also, the loop iteration variable must be an integer and the loop control parameters must be the same for all threads.

Program correctness must not depend upon which thread executes a particular iteration.

It is illegal to branch (goto) out of a loop associated with a DO/for directive.

The chunk size must be specified as a loop invariant integer expression, as there is no synchronization during its evaluation by different threads.

ORDERED and SCHEDULE clauses may appear once each.

Sample Code

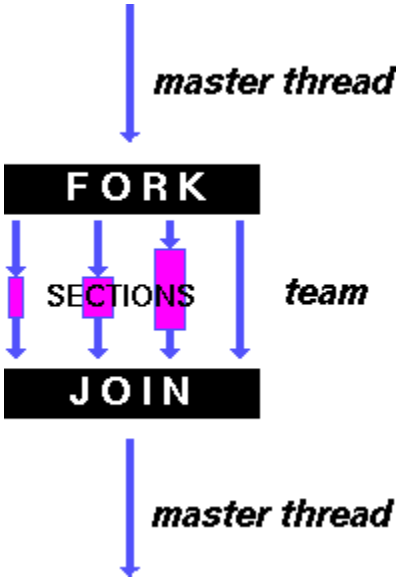
```
#include <omp.h>
#define CHUNKSIZE 100
#define N     1000

main ()
{

int i, chunk;
float a[N], b[N], c[N];

/* Some initializations */
for (i=0; i < N; i++)
  a[i] = b[i] = i * 1.0;
chunk = CHUNKSIZE;

#pragma omp parallel shared(a,b,c,chunk) private(i)
  {

  #pragma omp for schedule(dynamic,chunk) nowait
  for (i=0; i < N; i++)
    c[i] = a[i] + b[i];

  }  /* end of parallel section */

}
```

Types of Work-Sharing Constructs

SECTIONS

Sections breaks work into separate, discrete sections. Each section is executed by a thread.

- Can be used to implement a type of “functional parallelism”.



Section Work-Sharing Construct

The SECTIONS directive is a non-iterative work-sharing construct.

- It specifies that the enclosed section(s) of code are to be divided among the threads in the team.

Independent SECTION directives are nested within a SECTIONS directive. Each SECTION is executed once by a thread in the team.

- Different sections may be executed by different threads.
- It is possible for a thread to execute more than one section if it is quick enough and the implementation permits such.

There is an implied barrier at the end of a SECTIONS directive, unless the NOWAIT clause is used.

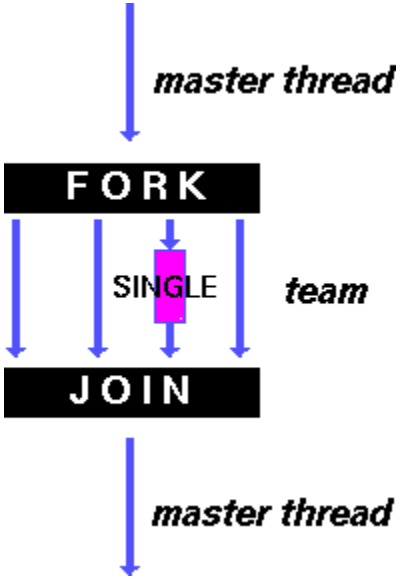
Sample Code

```
#include <omp.h>
#define N     1000

main ()
{
int i;
float a[N], b[N], c[N], d[N];

/* Some initializations */
for (i=0; i < N; i++) {
  a[i] = i * 1.5;
  b[i] = i + 22.35;
  }

#pragma omp parallel shared(a,b,c,d) private(i)
  {
  #pragma omp sections nowait
    {
    #pragma omp section
    for (i=0; i < N; i++)
      c[i] = a[i] + b[i];

    #pragma omp section
    for (i=0; i < N; i++)
      d[i] = a[i] * b[i];

    }  /* end of sections */
  }  /* end of parallel section */
}
```

Types of Work-Sharing Constructs

SINGLE

SINGLE serializes a section of code



Single Work-Sharing Construct

The SINGLE directive specifies that the enclosed code is to be executed by only one thread in the team.

Useful when dealing with sections of code that are not thread safe (such as I/O)

Threads in the team that do not execute the SINGLE directive, wait at the end of the enclosed code block, unless a NOWAIT/nowait clause is specified.

It is illegal to branch into or out of a SINGLE block.

Sample Code

```c
#include <stdio.h>
#include <omp.h>

int main() {
    omp_set_num_threads(4); // Set number of threads

    #pragma omp parallel
    {
        int thread_id = omp_get_thread_num();

        // Each thread prints its ID
        printf("Thread %d: Before SINGLE\n", thread_id);

        // Only one thread executes this block
        #pragma omp single
        {
            printf("Thread %d: Executing SINGLE block\n", thread_id);
        }

        // All threads wait at the implicit barrier (unless nowait is used)
        printf("Thread %d: After SINGLE\n", thread_id);
    }

    return 0;
```

Combined Parallel Work-Sharing Constructs

OpenMP provides three directives that are merely conveniences:

- PARALLEL DO / parallel for
- PARALLEL SECTIONS

For the most part, these directives behave identically to an individual PARALLEL directive being immediately followed by a separate work-sharing directive.

Most of the rules, clauses and restrictions that apply to both directives are in effect.

Sample Code

```
#include <omp.h>
#define N       1000
#define CHUNKSIZE   100

main ()  {

int i, chunk;
float a[N], b[N], c[N];

/* Some initializations */
for (i=0; i < N; i++)
  a[i] = b[i] = i * 1.0;
chunk = CHUNKSIZE;

#pragma omp parallel for \
   shared(a,b,c,chunk) private(i) \
   schedule(static,chunk)
  for (i=0; i < n; i++)
    c[i] = a[i] + b[i];
}
```